

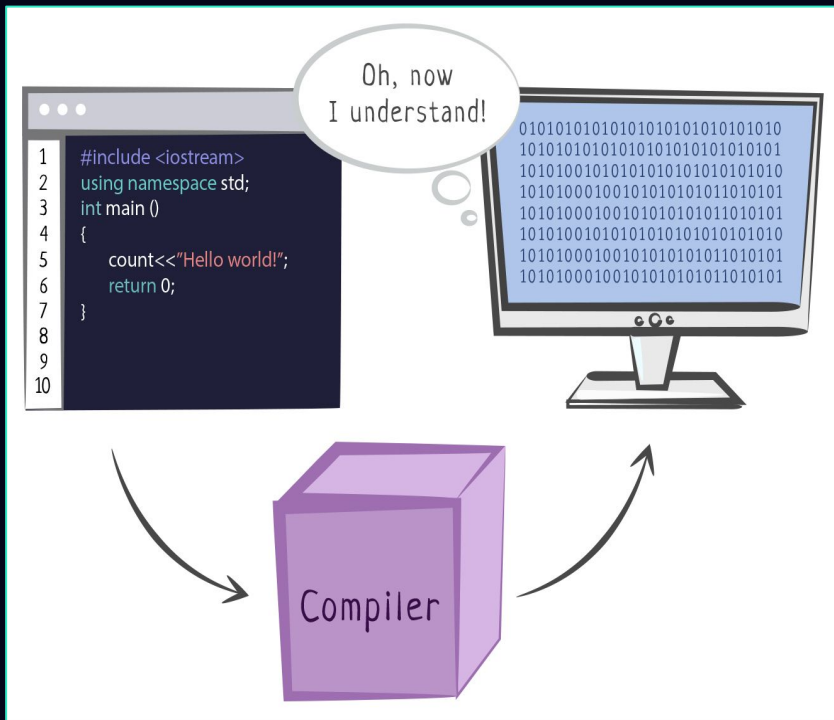
Сравнительный анализ JIT и AOT компиляции на примере LLVM и V8

Дворкин Борис Александрович Р3231

Процесс компиляции

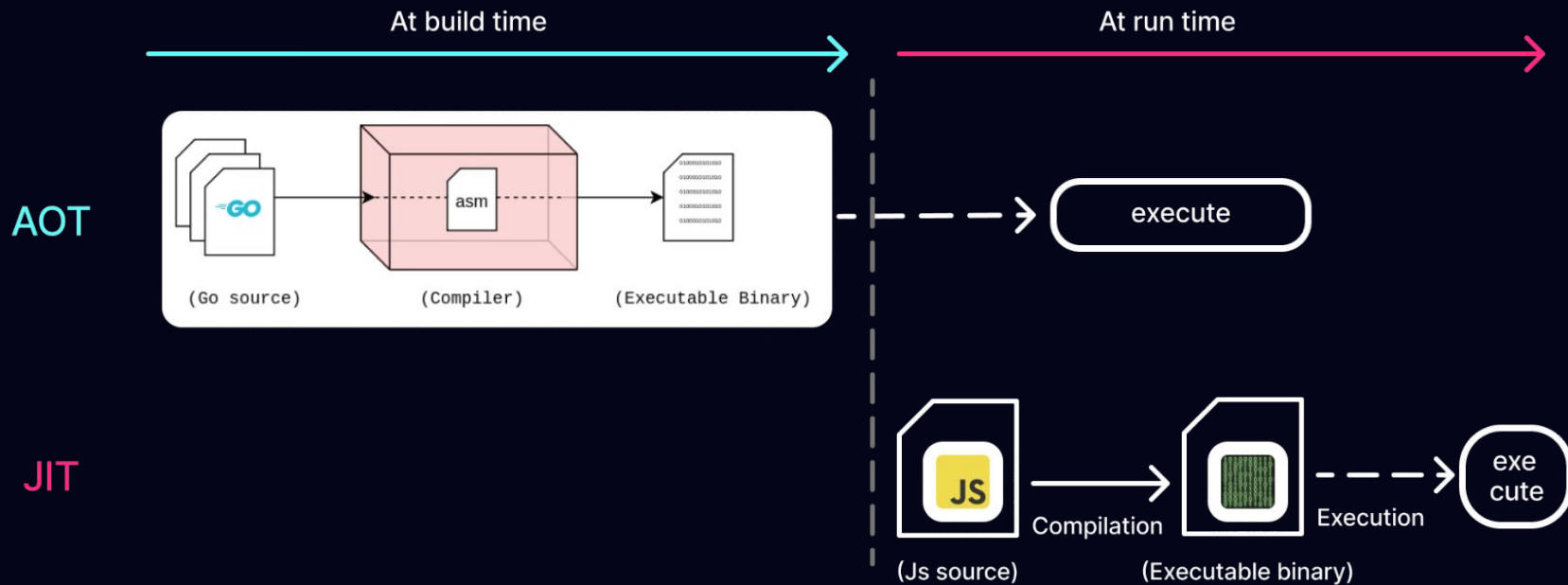
От простого к сложному

- Что такое компиляция?
- Понимание разницы между JIT и AOT компиляцией.
- Изучение процесса компиляции на примере LLVM и V8.

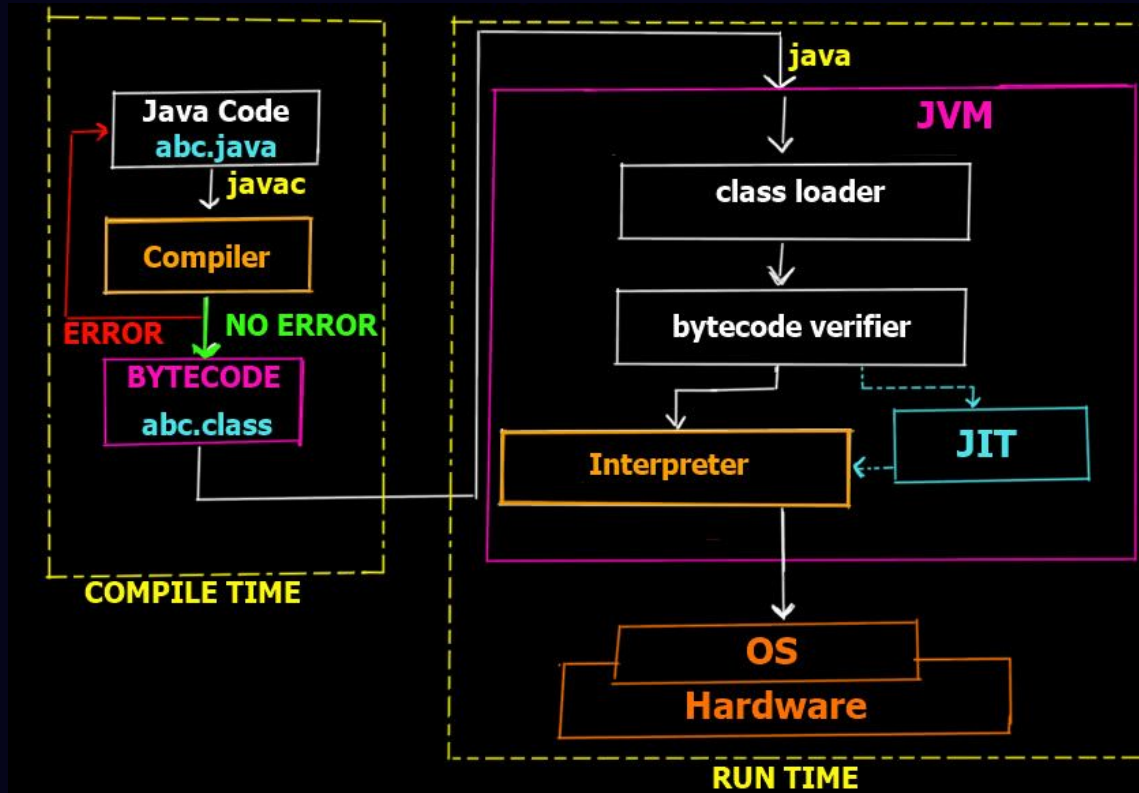


Время компиляции

*упрощение



Java compile process



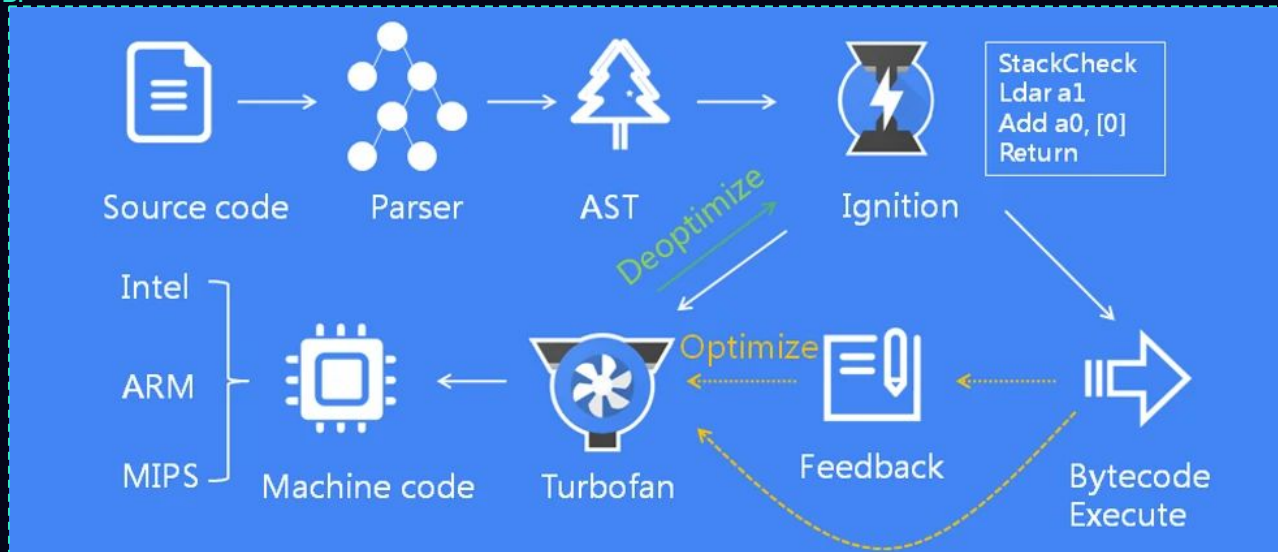
- HotSpot JVM (1999)
- Компилятор javac
- загрузчик class loader
- bytecode verifier
- JVM interpreter
- JVM JIT

V8 Javascript engine (Runtime env)

Диаграмма архитектуры

Основные элементы архитектуры

- Интерпретатор Ignition
- Jit-компилятор Turbofan
- Куча и стек вызовов
- Модули памяти
- Сборщик мусора



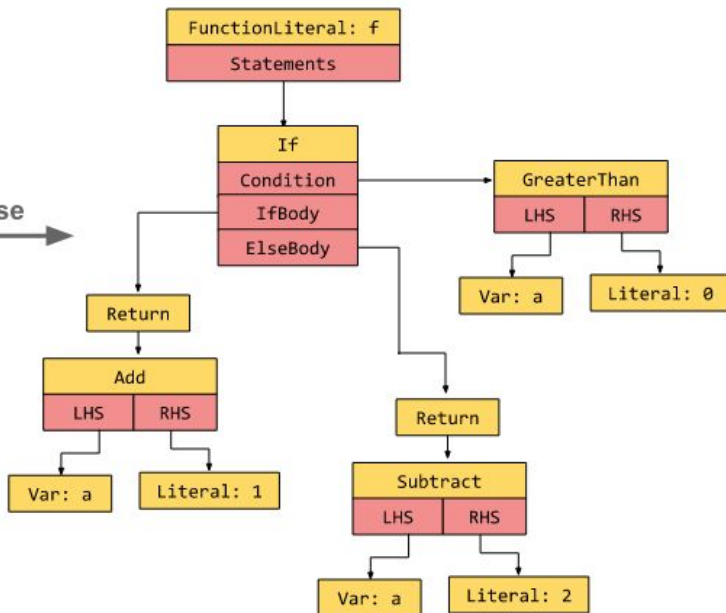
V8 Javascript engine

JavaScript Source

```
function f(a) {  
  if (a > 0) {  
    return a + 1;  
  } else {  
    return a - 2;  
  }  
}
```

Parse

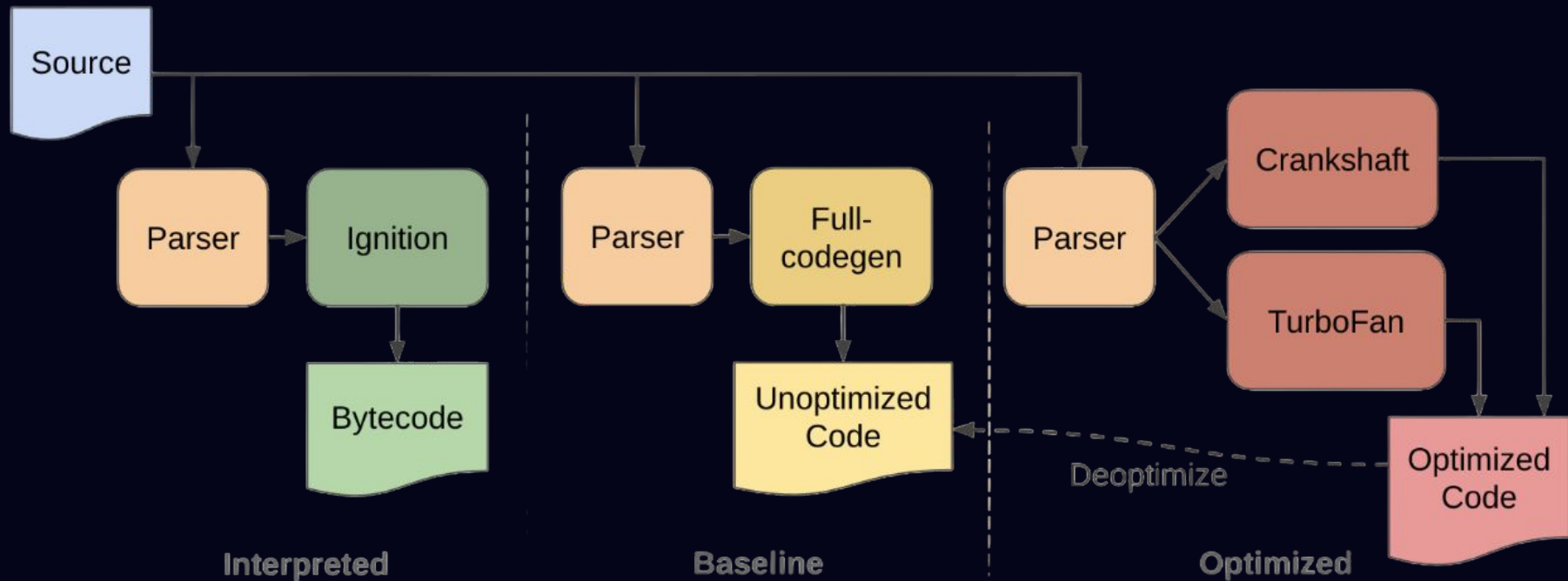
Abstract Syntax Tree



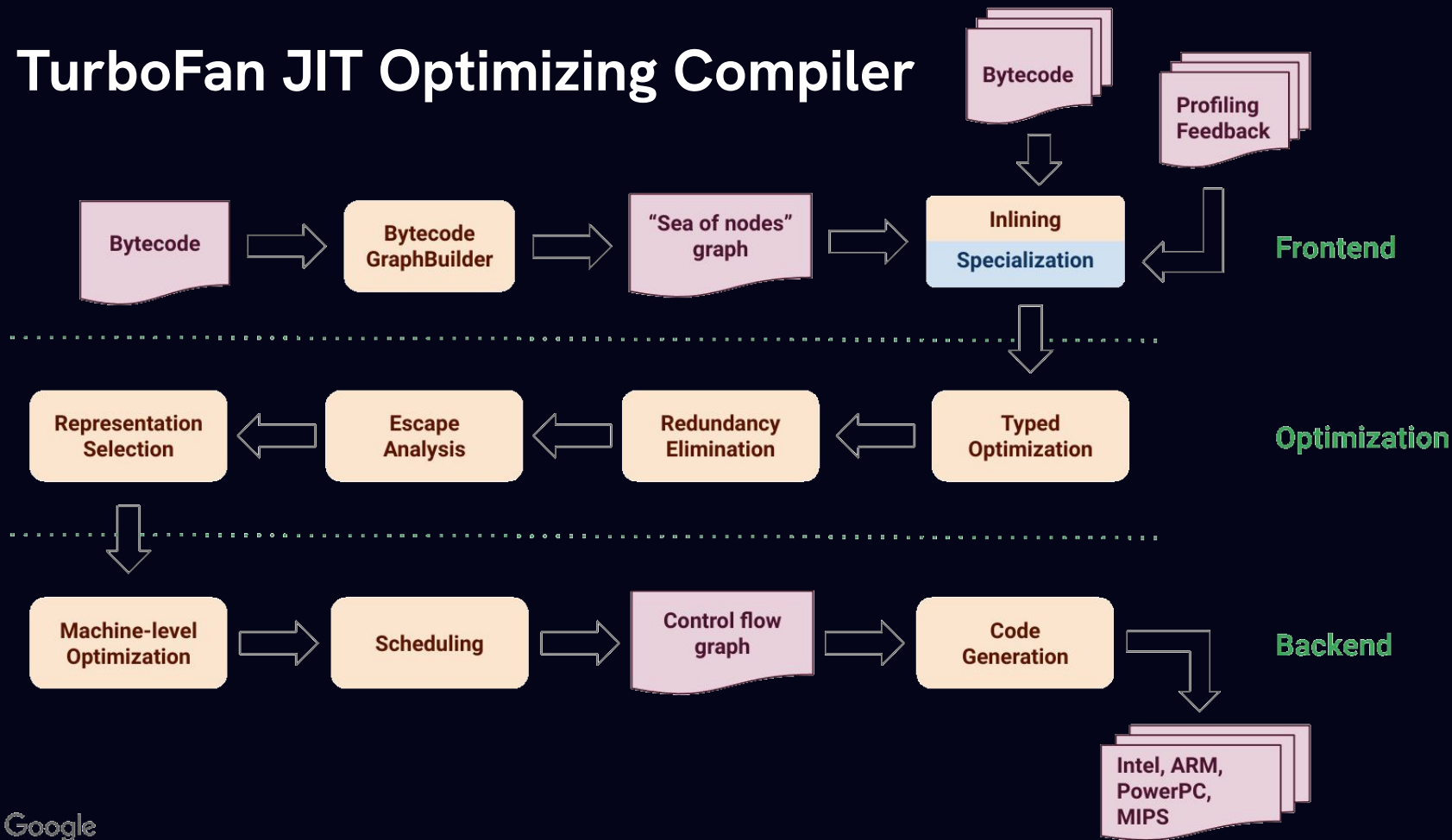
Ignition Bytecode

StackCheck
LdaZero
TestGreaterThan a0
JumpIfFalse [@else]
Ldar a0
AddSmi #1
Return
@else:
Ldar a0
SubSmi #2
Return

V8 Javascript compilation pipeline



TurboFan JIT Optimizing Compiler



Методы оптимизаций Turbofan JIT



Оптимизации

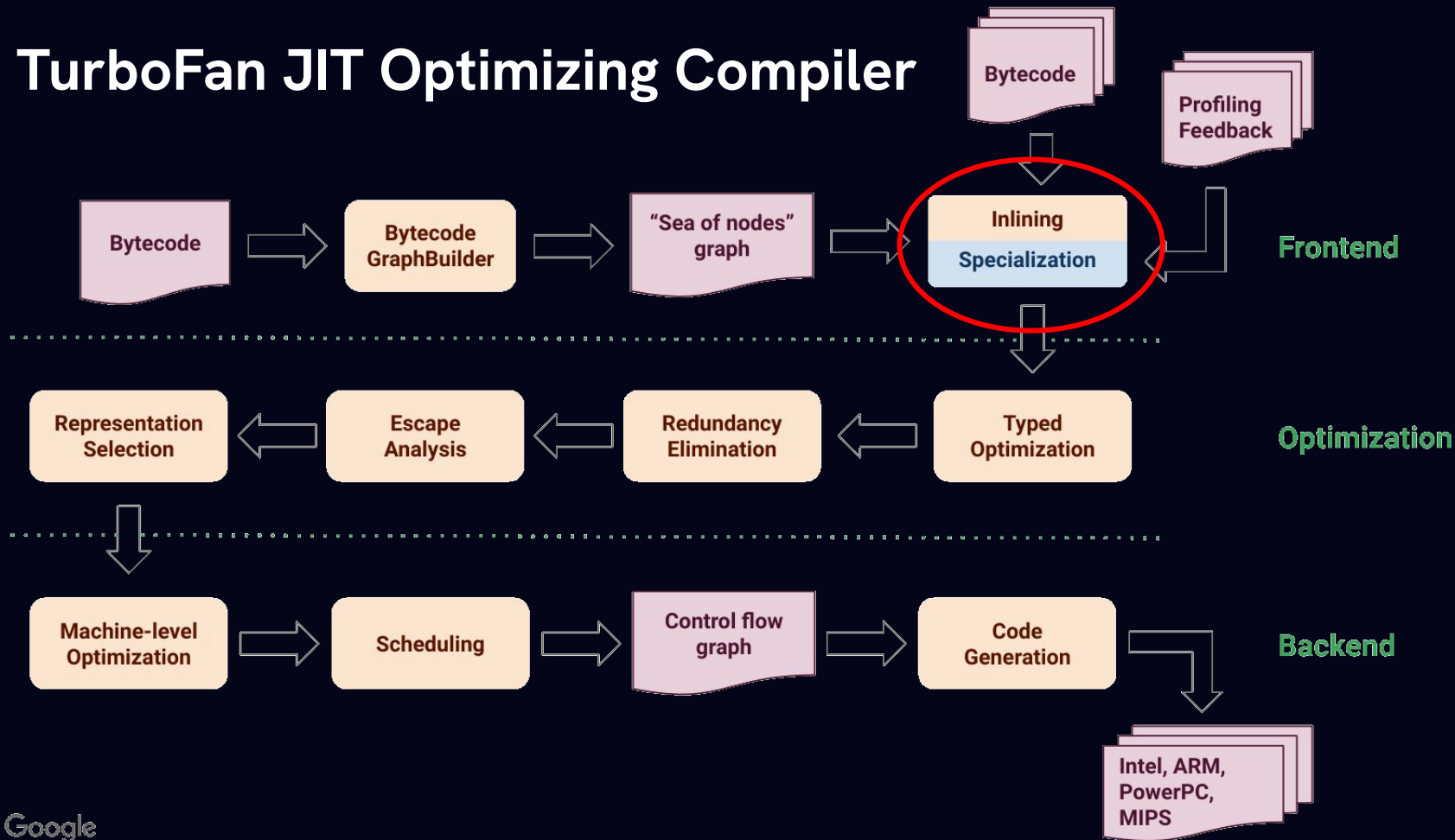
- Инлайнинг
- Удаление мертвого кода
- Свертывание констант
- Разворачивание циклов
- Анализ побега
- <...>



Вызовы и risks

- Риск ошибочных оптимизаций, требует ресурсы на "откат" машинного кода обратно в байт-код или перекомпиляцию
- Затраты на хранение маш.кода (memory consumption goes brrr)
- (Раньше был baseline компилятор, новый интерпретатор Ignition спасает)

TurboFan JIT Optimizing Compiler



Inlining

```
function add(x, y) {  
  return x + y;  
}
```

```
function three() {  
  return add(1, 2);  
}
```



```
function three_add_inlined() {  
  var x = 1;  
  var y = 2;  
  var add_return_value = x + y;  
  return add_return_value;  
}
```

Inlining

```
function add(x, y) {  
  return x + y;  
}
```

```
function three() {  
  return add(1, 2);  
}
```



```
function three_add_inlined() {  
  var x = 1;  
  var y = 2;  
  var add_return_value = x + y;  
  return add_return_value;  
}
```

*Constant-
folding*

```
function three_add_const_folded() {  
  return 3;  
}
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```



```
leaq rcx,[rip+0x0]  
movq rcx,[rcx-0x37]  
testb [rcx+0xf],0x1  
jnz CompileLazyDeoptimizedCode  
push rbp  
movq rbp, rsp  
push rsi  
push rdi  
cmpq rsp,[r13+0xdb0]  
jna StackCheck  
movq rax,[rbp+0x18]  
test al,0x1  
jnz Deoptimize  
movq rbx,[rbp+0x10]  
testb rbx,0x1  
jnz Deoptimize  
movq rdx,rbx  
shrq rdx, 32  
movq rcx,rax  
shrq rcx, 32  
addl rdx,rcx  
jo Deoptimize  
shlq rdx, 32  
movq rax,rdx  
movq rsp,rbp  
pop rbp  
ret 0x18
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue

```
    leaq rcx,[rip+0x0]  
    movq rcx,[rcx-0x37]  
    testb [rcx+0xf],0x1  
    jnz CompileLazyDeoptimizedCode  
    push rbp  
    movq rbp, rsp  
    push rsi  
    push rdi  
    cmpq rsp,[r13+0xdb0]  
    jna StackCheck  
    movq rax,[rbp+0x18]  
    test al,0x1  
    jnz Deoptimize  
    movq rbx,[rbp+0x10]  
    testb rbx,0x1  
    jnz Deoptimize  
    movq rdx,rbx  
    shrq rdx, 32  
    movq rcx,rax  
    shrq rcx, 32  
    addl rdx,rcx  
    jo Deoptimize  
    shlq rdx, 32  
    movq rax,rdx  
    movq rsp,rbp  
    pop rbp  
    ret 0x18
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue

Check x is a small integer

```
    leaq rcx,[rip+0x0]  
    movq rcx,[rcx-0x37]  
    testb [rcx+0xf],0x1  
    jnz CompileLazyDeoptimizedCode  
    push rbp  
    movq rbp, rsp  
    push rsi  
    push rdi  
    cmpq rsp,[r13+0xdb0]  
    jna StackCheck  
    movq rax,[rbp+0x18]  
    test al,0x1  
    jnz Deoptimize  
    movq rbx,[rbp+0x10]  
    testb rbx,0x1  
    jnz Deoptimize  
    movq rdx,rbx  
    shrq rdx, 32  
    movq rcx,rax  
    shrq rcx, 32  
    addl rdx,rcx  
    jo Deoptimize  
    shlq rdx, 32  
    movq rax,rdx  
    movq rsp,rbp  
    pop rbp  
    ret 0x18
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue

Check x is a small integer

Check y is a small integer

```
    leaq rcx,[rip+0x0]  
    movq rcx,[rcx-0x37]  
    testb [rcx+0xf],0x1  
    jnz CompileLazyDeoptimizedCode  
    push rbp  
    movq rbp, rsp  
    push rsi  
    push rdi  
    cmpq rsp,[r13+0xdb0]  
    jna StackCheck  
    movq rax,[rbp+0x18]  
    test al,0x1  
    jnz Deoptimize  
    movq rbx,[rbp+0x10]  
    testb rbx,0x1  
    jnz Deoptimize  
    movq rdx,rbx  
    shrq rdx, 32  
    movq rcx,rax  
    shrq rcx, 32  
    addl rdx,rcx  
    jo Deoptimize  
    shlq rdx, 32  
    movq rax,rdx  
    movq rsp,rbp  
    pop rbp  
    ret 0x18
```


Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Check x is a small integer

Check y is a small integer

Convert y from tagged representation to word32

Prologue

```
leaq rcx,[rip+0x0]  
movq rcx,[rcx-0x37]  
testb [rcx+0xf],0x1  
jnz CompileLazyDeoptimizedCode  
push rbp  
movq rbp, rsp  
push rsi  
push rdi  
cmpq rsp,[r13+0xdb0]  
jna StackCheck  
movq rax,[rbp+0x18]  
test al,0x1  
jnz Deoptimize  
movq rbx,[rbp+0x10]  
testb rbx,0x1  
jnz Deoptimize  
movq rdx,rbx  
shrq rdx, 32  
movq rcx,rax  
shrq rcx, 32  
addl rdx,rcx  
jo Deoptimize  
shlq rdx, 32  
movq rax,rdx  
movq rsp,rbp  
pop rbp  
ret 0x18
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Check x is a small integer

Check y is a small integer

Convert y from tagged representation to word32

Convert x from tagged representation to word32

Prologue

```
leaq rcx,[rip+0x0]  
movq rcx,[rcx-0x37]  
testb [rcx+0xf],0x1  
jnz CompileLazyDeoptimizedCode  
push rbp  
movq rbp, rsp  
push rsi  
push rdi  
cmpq rsp,[r13+0xdb0]  
jna StackCheck  
movq rax,[rbp+0x18]  
test al,0x1  
jnz Deoptimize  
movq rbx,[rbp+0x10]  
testb rbx,0x1  
jnz Deoptimize  
movq rdx,rbx  
shrq rdx, 32  
movq rcx, rax  
shrq rcx, 32  
addl rdx,rcx  
jo Deoptimize  
shlq rdx, 32  
movq rax,rdx  
movq rsp,rbp  
pop rbp  
ret 0x18
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue

Check x is a small integer

Check y is a small integer

Convert y from tagged representation to word32

Convert x from tagged representation to word32

Add x and y (incl. overflow check)

```
leaq rcx,[rip+0x0]  
movq rcx,[rcx-0x37]  
testb [rcx+0xf],0x1  
jnz CompileLazyDeoptimizedCode  
push rbp  
movq rbp,rsi  
push rsi  
push rdi  
cmpq rsp,[r13+0xdb0]  
jna StackCheck  
movq rax,[rbp+0x18]  
test al,0x1  
jnz Deoptimize  
movq rbx,[rbp+0x10]  
testb rbx,0x1  
jnz Deoptimize  
movq rdx,rbx  
shrq rdx, 32  
movq rcx,rax  
shrq rcx, 32  
addl rdx,rcx  
jo Deoptimize  
shlq rdx, 32  
movq rax,rdx  
movq rsp,rbp  
pop rbp  
ret 0x18
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue

Check x is a small integer

Check y is a small integer

Convert y from tagged representation to word32

Convert x from tagged representation to word32

Add x and y (incl. overflow check)

Convert result to tagged representation

```
leaq rcx,[rip+0x0]  
movq rcx,[rcx-0x37]  
testb [rcx+0xf],0x1  
jnz CompileLazyDeoptimizedCode  
push rbp  
movq rbp, rsp  
push rsi  
push rdi  
cmpq rsp,[r13+0xdb0]  
jna StackCheck  
movq rax,[rbp+0x18]  
test al,0x1  
jnz Deoptimize  
movq rbx,[rbp+0x10]  
testb rbx,0x1  
jnz Deoptimize  
movq rdx,rbx  
shrq rdx, 32  
movq rcx,rax  
shrq rcx, 32  
addl rdx,rcx  
jo Deoptimize  
shlq rdx, 32  
movq rax,rdx  
movq rsp,rbp  
pop rbp  
ret 0x18
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue

Check x is a small integer

Check y is a small integer

Convert y from tagged representation to word32

Convert x from tagged representation to word32

Add x and y (incl. overflow check)

Convert result to tagged representation

Epilogue

```
leaq rcx,[rip+0x0]  
movq rcx,[rcx-0x37]  
testb [rcx+0xf],0x1  
jnz CompileLazyDeoptimizedCode  
push rbp  
movq rbp, rsp  
push rsi  
push rdi  
cmpq rsp,[r13+0xdb0]  
jna StackCheck  
movq rax,[rbp+0x18]  
test al,0x1  
jnz Deoptimize  
movq rbx,[rbp+0x10]  
testb rbx,0x1  
jnz Deoptimize  
movq rdx,rbx  
shrq rdx, 32  
movq rcx,rax  
shrq rcx, 32  
addl rdx,rcx  
jo Deoptimize  
shlq rdx, 32  
movq rax,rdx  
movq rsp,rbp  
pop rbp  
ret 0x18
```

Inlining

```
function add(x, y) {  
  return x + y;  
}
```

```
function three() {  
  return add(1, 2);  
}
```



```
leaq rcx,[rip+0x0]  
movq rcx,[rcx-0x37]  
testb [rcx+0xf],0x1  
jnz CompileLazyDeoptimizedCode  
push rbp  
movq rbp,rsr  
push rsi  
push rdi  
cmpq rsp,[r13+0xdb0]  
jna StackCheck  
movq rdi,<JSFunction add>  
movq rsi,[rdi+0x1f]  
movq rax,<JSGlobal Object>  
push rax  
movq rax,0x100000000  
push rax  
movq rax,0x200000000  
push rax  
movq rdx,[r13-0x60]  
movl rax,0x2  
movq rcx,[rdi+0x2f]  
addq rcx,0x5f  
call rcx  
movq rsp,rbp  
pop rbp  
ret 0x8
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue

```
{  
    leaq rcx,[rip+0x0]  
    movq rcx,[rcx-0x37]  
    testb [rcx+0xf],0x1  
    jnz CompileLazyDeoptimizedCode  
    push rbp  
    movq rbp,rsi  
    push rsi  
    push rdi  
    cmpq rsp,[r13+0xdb0]  
    jna StackCheck  
    movq rdi,<JSFunction add>  
    movq rsi,[rdi+0x1f]  
    movq rax,<JSGlobal Object>  
    push rax  
    movq rax,0x100000000  
    push rax  
    movq rax,0x200000000  
    push rax  
    movq rdx,[r13-0x60]  
    movl rax,0x2  
    movq rcx,[rdi+0x2f]  
    addq rcx,0x5f  
    call rcx  
    movq rsp,rbp  
    pop rbp  
    ret 0x8  
}
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue

Load call target add

```
leaq rcx,[rip+0x0]  
movq rcx,[rcx-0x37]  
testb [rcx+0xf],0x1  
jnz CompileLazyDeoptimizedCode  
push rbp  
movq rbp,rsr  
push rsi  
push rdi  
cmpq rsp,[r13+0xdb0]  
jna StackCheck  
movq rdi,<JSFunction add>  
movq rsi,[rdi+0x1f]  
movq rax,<JSGlobal Object>  
push rax  
movq rax,0x100000000  
push rax  
movq rax,0x200000000  
push rax  
movq rdx,[r13-0x60]  
movl rax,0x2  
movq rcx,[rdi+0x2f]  
addq rcx,0x5f  
call rcx  
movq rsp,rbp  
pop rbp  
ret 0x8
```


Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue

Load call target add

Load call parameters

```
leaq rcx,[rip+0x0]  
movq rcx,[rcx-0x37]  
testb [rcx+0xf],0x1  
jnz CompileLazyDeoptimizedCode  
push rbp  
movq rbp,rsq  
push rsi  
push rdi  
cmpq rsp,[r13+0xdb0]  
jna StackCheck  
movq rdi,<JSFunction add>  
movq rsi,[rdi+0x1f]  
movq rax,<JSGlobal Object>  
push rax  
movq rax,0x100000000  
push rax  
movq rax,0x200000000  
push rax  
movq rdx,[r13-0x60]  
movl rax,0x2  
movq rcx,[rdi+0x2f]  
addq rcx,0x5f  
call rcx  
movq rsp,rbp  
pop rbp  
ret 0x8
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue

Load call target add

Load call parameters

Call sequence

```
leaq rcx,[rip+0x0]  
movq rcx,[rcx-0x37]  
testb [rcx+0xf],0x1  
jnz CompileLazyDeoptimizedCode  
push rbp  
movq rbp,rsr  
push rsi  
push rdi  
cmpq rsp,[r13+0xdb0]  
jna StackCheck  
movq rdi,<JSFunction add>  
movq rsi,[rdi+0x1f]  
movq rax,<JSGlobal Object>  
push rax  
movq rax,0x100000000  
push rax  
movq rax,0x200000000  
push rax  
movq rdx,[r13-0x60]  
movl rax,0x2  
movq rcx,[rdi+0x2f]  
addq rcx,0x5f  
call rcx  
movq rsp,rbp  
pop rbp  
ret 0x8
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue	<pre>leaq rcx,[rip+0x0] movq rcx,[rcx-0x37] <u>testb [rcx+0xf],0x1</u> <u>jnz CompileLazyDeoptimizedCode</u> push rbp movq rbp,rsr push rsi push rdi cmpq rsp,[r13+0xdb0] jna StackCheck</pre>
Load call target <u>add</u>	<pre>movq rdi,<JSFunction add> movq rsi,[rdi+0x1f] movq rax,<<u>JSGlobal</u> Object> push rax</pre>
Load call parameters	<pre>movq rax,0x100000000 push rax movq rax,0x200000000 push rax movq rdx,[r13-0x60] movl rax,0x2</pre>
Call sequence	<pre>movq rcx,[rdi+0x2f] addq rcx,0x5f call rcx</pre>
Epilogue	<pre>movq rsp,rbp pop rbp ret 0x8</pre>

Inlining

```
function add(x, y) {  
  return x + y;  
}
```

```
function three() {  
  return add(1, 2);  
}
```

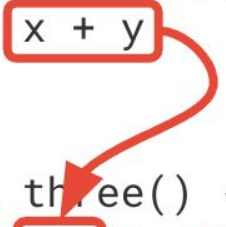


```
leaq rcx,[rip+0x0]  
movq rcx,[rcx-0x37]  
testb [rcx+0xf],0x1  
jnz CompileLazyDeoptimizedCode  
push rbp  
movq rbp,rsi  
push rdi  
cmpq rsp,[r13+0xdb0]  
jna StackCheck  
movq rax,0x300000000  
movq rsp,rbp  
pop rbp  
ret 0x8
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```



Prologue

```
{  
    leaq rcx,[rip+0x0]  
    movq rcx,[rcx-0x37]  
    testb [rcx+0xf],0x1  
    jnz CompileLazyDeoptimizedCode  
    push rbp  
    movq rbp, rsp  
    push rsi  
    push rdi  
    cmpq rsp,[r13+0xdb0]  
    jna StackCheck  
    movq rax,0x300000000  
    movq rsp, rbp  
    pop rbp  
    ret 0x8  
}
```

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Prologue

```
{  
    leaq rcx,[rip+0x0]  
    movq rcx,[rcx-0x37]  
    testb [rcx+0xf],0x1  
    jnz CompileLazyDeoptimizedCode  
    push rbp  
    movq rbp, rsp  
    push rsi  
    push rdi  
    cmpq rsp,[r13+0xdb0]  
    jna StackCheck  
    movq rax,0x300000000  
    movq rsp, rbp  
    pop rbp  
    ret 0x8  
}
```

Load tagged value 3

Inlining

```
function add(x, y) {  
    return x + y;  
}
```

```
function three() {  
    return add(1, 2);  
}
```

Load tagged value 3

Prologue

Epilogue

```
{  
    leaq rcx,[rip+0x0]  
    movq rcx,[rcx-0x37]  
    testb [rcx+0xf],0x1  
    jnz CompileLazyDeoptimizedCode  
    push rbp  
    movq rbp, rsp  
    push rsi  
    push rdi  
    cmpq rsp,[r13+0xdb0]  
    jna StackCheck  
    movq rax,0x300000000  
    movq rsp,rbp  
    pop rbp  
    ret 0x8  
}
```

Inlining

```
leaq rcx,[rip+0x0]
movq rcx,[rcx-0x37]
testb [rcx+0xf],0x1
jnz CompileLazyDeoptimizedCode
push rbp
movq rbp, rsp
push rsi
push rdi
cmpq rsp,[r13+0xdb0]
jna StackCheck
movq rdi,<JSFunction add>
movq rsi,[rdi+0x1f]
movq rax,<JSGlobal Object>
push rax
movq rax,0x100000000
push rax
movq rax,0x200000000
push rax
movq rdx,[r13-0x60]
movl rax,0x2
movq rcx,[rdi+0x2f]
addq rcx,0x5f
call rcx
movq rsp, rbp
pop rbp
ret 0x8
```

+

```
leaq rcx,[rip+0x0]
movq rcx,[rcx-0x37]
testb [rcx+0xf],0x1
jnz CompileLazyDeoptimizedCode
push rbp
movq rbp, rsp
push rsi
push rdi
cmpq rsp,[r13+0xdb0]
jna StackCheck
movq rax,[rbp+0x18]
test al,0x1
jnz Deoptimize
movq rbx,[rbp+0x10]
testb rbx,0x1
jnz Deoptimize
movq rdx,rbx
shrq rdx, 32
movq rcx,rax
shrq rcx, 32
addl rdx,rcx
jo Deoptimize
shlq rdx, 32
movq rax,rdx
movq rsp,rbp
pop rbp
ret 0x18
```

VS

```
leaq rcx,[rip+0x0]
movq rcx,[rcx-0x37]
testb [rcx+0xf],0x1
jnz CompileLazyDeoptimizedCode
push rbp
movq rbp, rsp
push rsi
push rdi
cmpq rsp,[r13+0xdb0]
jna StackCheck
movq rax,0x300000000
movq rsp,rbp
pop rbp
ret 0x8
```


Jit-компиляторы



HHVM обеспечивает JIT-компиляцию для PHP и Hack, оптимизируя выполнение кода на лету.



LLVM's MCJIT используется для JIT-компиляции LLVM IR (вместо статического бэкенда компилятора)



.NET JIT компилирует MSIL (Microsoft intermediate language) в машинный код.



SpiderMonkey engine: Baseline hybrid interpreter/JIT + Baseline Compiler + WarpMonkey JIT



TurboFan JIT - часть V8, оптимизирует работу интерпретатора Ignition

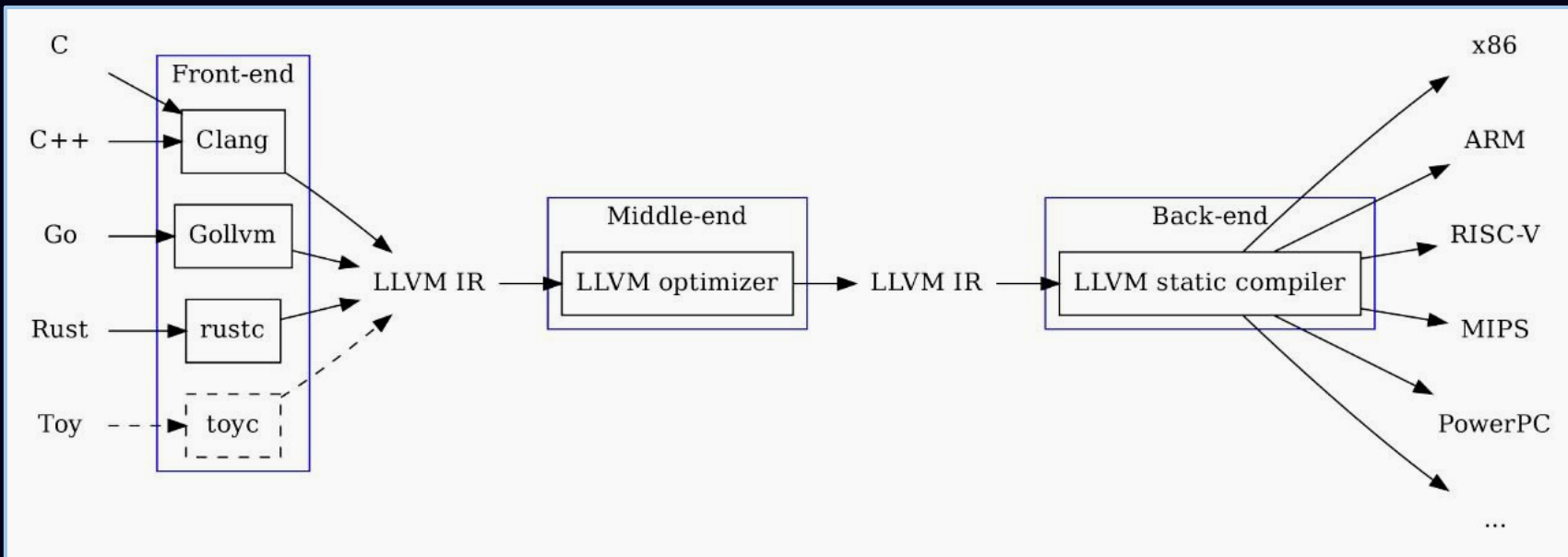


C1 & C2 - житы в HotSpot JVM. C1 мгновенно компилирует, C2 делает более глубокие оптимизации

(AoT) Ahead-of-Time compilation

Компоненты
компилятора

- Фронтенд
- Мидлэнд
- Бэкенд



LLVM для создания своего языка

1

Придумываем язык

2

Пишем фронтенд - компилятор который будет делать LLVM IR из вашего языка

3

Всю остальную работу на оптимизации и получение машинного кода целевой платформы отдаём тулчейну LLVM

4

Можно также отредактировать бэкенд чтобы получить машинный код своей специфической платформы

Методы оптимизации LLVM



Оптимизации LLVM

- Инлайнинг функций
- Удаление мертвого кода
- Свертывание констант
- Разворачивание циклов
- Оптимизация хвостовых вызовов



Преимущества оптимизаций LLVM

- Применяются на различных этапах компиляции для улучшения производительности
- Уменьшают размер и сложность сгенерированного кода
- Повышают эффективность работы с памятью

```
int factorial(int n) {
    if (n == 1) return 1;
    return n * factorial(n - 1);
}

int main() {
    return factorial(5);
    return factorial(5);
}
```

C source code

Clang

LLVM IR

```
define dso_local i32 @factorial(i32 @noundef %0) #0 !dbg !10 {
    %2 = alloca i32, align 4
    %3 = alloca i32, align 4
    store i32 %0, ptr %3, align 4
    call void @llvm.dbg.declare(metadata ptr %3, metadata !16, metadata !DIExpression()), !dbg !17
    %4 = load i32, ptr %3, align 4, !dbg !18
    %5 = icmp eq i32 %4, 1, !dbg !20
    br i1 %5, label %6, label %7, !dbg !21
}

6:
    store i32 1, ptr %2, align 4, !dbg !22
    br label %13, !dbg !22

7:
    %8 = load i32, ptr %3, align 4, !dbg !23
    %9 = load i32, ptr %3, align 4, !dbg !24
    %10 = sub nsw i32 %9, 1, !dbg !25
    %11 = call i32 @factorial(i32 @noundef %10), !dbg !26
    %12 = mul nsw i32 %8, %11, !dbg !27
    store i32 %12, ptr %2, align 4, !dbg !28
    br label %13, !dbg !28

13:
    %14 = load i32, ptr %2, align 4, !dbg !29
    ret i32 %14, !dbg !29
}

declare void @llvm.dbg.declare(metadata, metadata, metadata) #1

define dso_local i32 @main() #0 !dbg !30 {
    %1 = alloca i32, align 4
    store i32 0, ptr %1, align 4
    %2 = call i32 @factorial(i32 @noundef 5), !dbg !33
    ret i32 %2, !dbg !34
}
```

x86-64 clang 18.1.0 -S -emit-llvm -Ofast

```
define dso_local i32 @factorial(i32 noundef %0) local_unnamed_addr #0 !dbg !10 {  
    tail call void @llvm.dbg.value(metadata i32 %0, metadata !16, metadata !DIExpr  
    %2 = icmp eq i32 %0, 1, !dbg !18  
    br i1 %2, label %36, label %3, !dbg !20  
  
3:  
    %4 = add i32 %0, -1, !dbg !20  
    %5 = icmp ult i32 %4, 8, !dbg !20  
    br i1 %5, label %27, label %6, !dbg !20  
  
6:  
    %7 = and i32 %4, -8, !dbg !20  
    %8 = sub i32 %0, %7, !dbg !20  
    %9 = insertelement <4 x i32> poison, i32 %0, i64 0, !dbg !20  
    %10 = shufflevector <4 x i32> %9, <4 x i32> poison, <4 x i32> zeroinitializer,  
    %11 = add <4 x i32> %10, <i32 0, i32 -1, i32 -2, i32 -3>, !dbg !20  
    br label %12, !dbg !20  
  
12:  
    %13 = phi i32 [ 0, %6 ], [ %20, %12 ]  
    %14 = phi <4 x i32> [ %11, %6 ], [ %21, %12 ]  
    %15 = phi <4 x i32> [ <i32 1, i32 1, i32 1, i32 1>, %6 ], [ %18, %12 ]  
    %16 = phi <4 x i32> [ <i32 1, i32 1, i32 1, i32 1>, %6 ], [ %19, %12 ]  
    %17 = add <4 x i32> %14, <i32 -4, i32 -4, i32 -4, i32 -4>  
    %18 = mul <4 x i32> %14, %15, !dbg !21  
    %19 = mul <4 x i32> %17, %16, !dbg !21  
    %20 = add nuw i32 %13, 8  
    %21 = add <4 x i32> %14, <i32 -8, i32 -8, i32 -8, i32 -8>  
    %22 = icmp eq i32 %20, %7  
    br i1 %22, label %23, label %12  
  
23:  
    %24 = mul <4 x i32> %19, %18, !dbg !20  
    %25 = tail call i32 @llvm.vector.reduce.mul.v4i32(<4 x i32> %24), !dbg !20  
    %26 = icmp eq i32 %4, %7, !dbg !20  
    br i1 %26, label %36, label %27, !dbg !20
```

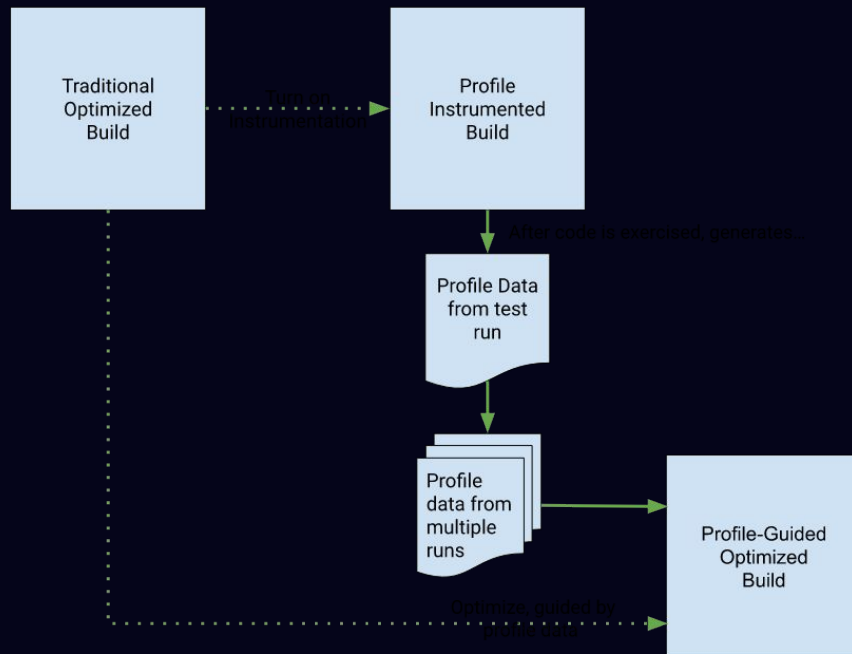
```
27:  
    %28 = phi i32 [ %0, %3 ], [ %8, %23 ]  
    %29 = phi i32 [ 1, %3 ], [ %25, %23 ]  
    br label %30, !dbg !20  
  
30:  
    %31 = phi i32 [ %33, %30 ], [ %28, %27 ]  
    %32 = phi i32 [ %34, %30 ], [ %29, %27 ]  
    tail call void @llvm.dbg.value(metadata i32 %31, metadata !16, meta  
    %33 = add nsw i32 %31, -1, !dbg !25  
    %34 = mul nsw i32 %31, %32, !dbg !21  
    tail call void @llvm.dbg.value(metadata i32 %33, metadata !16, meta  
    %35 = icmp eq i32 %33, 1, !dbg !18  
    br i1 %35, label %36, label %30, !dbg !20  
  
36:  
    %37 = phi i32 [ 1, %1 ], [ %25, %23 ], [ %34, %30 ], !dbg !21  
    ret i32 %37, !dbg !27  
}  
  
define dso_local noundef i32 @main() local_unnamed_addr #1 !dbg !28 {  
    ret i32 120, !dbg !31  
}  
  
declare void @llvm.dbg.value(metadata, metadata, metadata) #2  
  
declare i32 @llvm.vector.reduce.mul.v4i32(<4 x i32>) #2
```

АоТ оптимизации - PGO

От обычной сборки до
Profile-guided-optimization

- Шаг 1: Обычная сборка приложения
- Шаг 2: Инструментированная сборка позволяющая получать данные профиля при помощи PGO инструментов
- Шаг 3: Профилирование типичных сценариев, сбор данных
- Шаг 4: Сборка, оптимизированная с учётом данных профиля

Диаграмма профилированной оптимизации сборки



Сравнение JIT и AOT



Производительность

Различия в подходах к оптимизации кода. Как JIT оптимизирует 'горячие' участки кода в процессе исполнения.



Скорость разработки

JIT позволяет протестировать код не отходя от кассы, Aot может требовать длительной сборки, а время есть не всегда



Безопасность

Влияние JIT на динамически типизированные языки. Преимущества и особенности оптимизации.



Ресурсы

JIT-компиляция может требовать значительных ресурсов во время выполнения, особенно хранение профилей

[<25min] Возможные будущие направления для JIT и AOT

AOT и производительность

Исследование влияния AOT компиляции на производительность и потребление ресурсов в различных типах приложений и устройствах (Native Image AOT)

Новые методы оптимизации

Разработка новых методов оптимизации для JIT компиляторов, направленных на повышение эффективности кода и уменьшение задержек в работе приложений.

Интеграция с фреймворками

Интеграция GraalVM Native Image в широко используемые фреймворки и платформы для ускорения запуска приложений и снижения потребления памяти.

[<25min] Profile-Guided Optimization vs JIT

+ Плюсы

- Профилирование в JIT компиляции помогает быстрее оптимизировать 'горячие' участки кода.
- JIT компиляция позволяет быстрее реагировать на изменения в коде.
- Возможность динамической оптимизации, что особенно полезно для интерпретируемых языков.

× Минусы

- Отсутствие необходимости во время выполнения для оптимизации кода.
- Возможность сокращения времени запуска приложений за счет предварительного анализа.
- Улучшенная производительность за счет более точного прогнозирования поведения кода.

[<25min] JIT в Java vs GraalVM Native image AoT

JIT в Java

- Стандартный подход в Java, использующийся без GraalVM, включает компиляцию Java-кода в байткод на этапе компиляции, а затем использование JIT во время выполнения для перевода в машинный код.
- JIT в Java обеспечивает динамическую оптимизацию кода, что повышает эффективность исполнения.
- Профилирование в JIT компиляции помогает оптимизировать 'горячие' участки кода.

GraalVM

- GraalVM предлагает альтернативу традиционному JIT, повышая производительность и эффективность.
- Она включает JIT-компилятор, который может работать как с JVM, так и с другими языками, что делает его более универсальным.
- GraalVM также поддерживает AOT компиляцию, что снижает задержки запуска приложений.

Вопрос к экзамену



Первый вопрос

Что такое профилирование в JIT компиляции и как оно помогает в оптимизации 'горячих' участков кода?



Второй вопрос

Какие ключевые методы оптимизации использует LLVM при Ahead-Of-Time компиляции статически типизированных языков, таких как C или Rust?